Prof. Dr. Hélio Pedrini

MC202 - Estruturas de Dados - 2s2026
Laboratório 03

UNICAMP

- **Prazo para submissão: 13/10/2026**

Fila de Triagem do Pronto-Socorro

Atividade

No pronto-socorro de um hospital, os pacientes são cadastrados na recepção à medida que chegam. Para cada paciente, são registrados o **nome**, a **idade** e a **gravidade** do seu quadro, representada por um número inteiro de 1 (menos grave) a 10 (mais grave).

A coordenação do pronto-socorro define qual critério de prioridade será usado para organizar a fila de atendimento:

- **G**: pacientes com **maior gravidade** são atendidos primeiro;
- **I**: pacientes com **maior idade** são atendidos primeiro.

Em ambos os critérios, a fila é organizada em ordem decrescente do valor escolhido. Quando dois pacientes empatam no critério de prioridade, mantém-se entre eles a **ordem de cadastro** na recepção, isto é, quem foi cadastrado antes é atendido primeiro.

A tabela a seguir mostra um grupo de pacientes, na ordem de cadastro, e a posição de cada um na fila quando o critério escolhido é a gravidade (**G**).

Nome	Idade	Gravidade	Posição na Fila
Ana	34	3	4
Bruno	71	9	1
Carla	25	9	2
Davi	60	6	3
Eva	71	3	5

Bruno e Carla empatam com gravidade 9, e Bruno é atendido primeiro por ter sido cadastrado antes. Pelo mesmo motivo, Ana é atendida antes de Eva, pois ambas possuem gravidade 3.

Sua tarefa consiste em organizar a fila de atendimento utilizando o algoritmo **Bubble Sort** e informar a ordem em que os pacientes serão atendidos.

Entrada

A primeira linha da entrada contém um número inteiro K, indicando a quantidade de casos de teste. Cada caso de teste é estruturado da seguinte forma:

- Uma linha com um inteiro N ($1 \leq N \leq 100$), a quantidade de pacientes, e um caractere C, o critério de prioridade, que pode ser `G` (gravidade) ou `I` (idade), separados por um espaço em branco.
- Seguem N linhas, na ordem de cadastro, cada uma contendo o nome do paciente, sua idade ($0 \leq$ idade ≤ 120) e sua gravidade ($1 \leq$ gravidade ≤ 10), separados por um espaço em branco. O nome é formado apenas por letras, sem espaços, e possui no máximo 30 caracteres. Pacientes diferentes podem ter o mesmo nome.

Saída

Para cada caso de teste, o programa deve gerar N linhas, cada uma contendo o nome de um paciente, na ordem em que os pacientes serão atendidos. Após os nomes de cada caso de teste, deve ser impressa uma linha em branco, inclusive após o último caso de teste.

Exemplo

Entrada

```
2
4 I
Joao 45 7
Lia 80 2
Rui 45 10
Sofia 12 7
5 G
Pedro 30 4
Marta 55 8
Tiago 19 8
Pedro 67 1
Luana 40 10
```

Saída

```
Lia
Joao
Rui
Sofia

Luana
Marta
Tiago
Pedro
Pedro

```

Explicação

No primeiro caso, a fila é organizada pela idade. Joao e Rui empatam com 45 anos, e Joao é atendido primeiro por ter sido cadastrado antes. No segundo caso, a fila é organizada pela gravidade. Marta e Tiago empatam com gravidade 8 e mantêm a ordem de cadastro. Os dois pacientes chamados Pedro são pessoas diferentes: o de gravidade 4 é atendido antes do outro com gravidade 1.

Orientações Importantes

- É preciso tomar cuidado com a formatação da saída, pois ela é comparada linha a linha com a saída esperada. Observe que cada caso de teste termina com uma linha em branco, inclusive o último.
- A ordenação deve ser feita obrigatoriamente com o algoritmo **bubble sort**, implementado por você. Não é permitido utilizar a função `qsort` ou outras funções de ordenação de bibliotecas.
- As idades e as gravidades devem ser armazenadas em vetores alocados dinamicamente com `malloc`, de acordo com o valor de N de cada caso de teste.
- A ordenação deve reorganizar os próprios dados dos pacientes. Sempre que dois pacientes trocarem de posição, o nome, a idade e a gravidade de ambos devem ser trocados, para que cada posição continue representando um único paciente.
- A ordenação deve ser implementada em uma função `bubbleSort`, com tipo de retorno `void`, definida fora da função `main`. A função deve reorganizar diretamente os dados recebidos como parâmetros.
- É **obrigatório** implementar as trocas por meio de funções que recebem ponteiros, definidas fora da função `main`, com os seguintes protótipos:

```
1 void swap(int *a, int *b);
2 void swapName(char *a, char *b);
```

  A função `swap` deve ser usada para trocar idades e gravidades, e a função `swapName` deve ser usada para trocar nomes.